

AI-ASSISTED CODING FOR PYTHON

Built live with an AI coding agent in VS Code

Cloud Kitchen Inventory Simulation

Mentor Demo Guide

2-Hour Live Session · Multi-Brand Cloud Kitchen Case Study

Python simulation built by prompting, reviewing, and testing an AI coding agent

For Instructor Use — every step, every prompt, and how to review what the AI produces

Proprietary content. © Great Learning. All Rights Reserved.

How to use this guide

This is an AI-ASSISTED CODING session. You do not hand-write the simulation — you build the entire cloud kitchen inventory system by prompting an AI coding agent inside VS Code, then reviewing and refining what it generates. The cloud kitchen project is the vehicle; the skill being taught is how to drive an AI coding agent well.

Each phase gives you: (1) the concept in plain English, (2) the exact PROMPT to give the agent, (3) how to REVIEW its output before accepting it, (4) the AI WATCH-OUT — where the agent tends to make mistakes, (5) the output you should see, and (6) a mentor script and discussion prompt.

Each phase is tagged BUILD LIVE (prompt the agent in front of learners) or SHOW PRE-BUILT (display the already-generated result while explaining the prompt that made it). In a 2-hour session you cannot generate every enhancement live — Section 0.3 explains the strategy.

- **Complete Section 0 the day before** — it includes VS Code setup and a mandatory dry run where you actually run every prompt once.
- **Read Section 3 (The AI-Assisted Coding Workflow) carefully** — it is the real subject of the session, not the inventory model.
- **When live output differs from this guide, quote your live output** (the AI generates code fresh, so minor variation is normal) — but the AI WATCH-OUT items are non-negotiable correctness checks.

0. Pre-Session Preparation & Agenda

Complete everything in this section at least one day before the live session. In an AI-assisted coding demo the biggest risk is the agent behaving differently live than in prep — so the dry run, where you actually run every prompt once, is non-negotiable.

0.1 Environment Checklist — VS Code, the AI Agent, and Python

- 1. Install VS Code (latest).** Install your AI coding agent extension (e.g. Codex / Copilot Chat / equivalent) from the Marketplace and sign in with your authorised account. Confirm the agent panel opens and is connected — locate it now, not in front of learners.
- 2. Confirm the agent can act on the workspace.** Open an empty folder, give a trivial prompt like 'create a file hello.py that prints hello', and confirm it proposes a file edit you can review and accept. This proves the agentic edit-and-review loop works on your machine.
- 3. Confirm Python 3.10+ is installed:** `python --version`. Select that interpreter in VS Code (Command Palette → Python: Select Interpreter).
- 4. Place seed_data.py** in the project folder. This is the one file you provide; everything else — `main.py`, tests, and refinements — gets generated by the agent during the session. `seed_data.py` contains 5 tables: Recipes (5 items), Inventory (14 ingredients), Orders (5 orders across 2 brands), Restock (5 flagged ingredients), and Status (delivery status per order).
- 5. Prepare your context:** either create the `PROJECT_SPEC.md` file (Section 3.3) or have the business brief and rules ready to paste into your first prompt. Either way, the restock threshold, par level, and 'one task at a time' rules must be stated up front.
- 6. DRY RUN (mandatory):** run the ENTIRE session once on your own — give each step's prompt to the agent, review what it generates, run it, and tick every item in the 'What You Should See' boxes against your own outputs. Save the generated `main.py` and test files as a known-good fallback (Strategy B). Note where the agent needed a refinement prompt and bake that into your live run.
- 7. Screen setup:** zoom VS Code and the integrated terminal to a readable size for the back of the room (Ctrl/Cmd +). Increase the agent panel font if possible. Close email/chat/notifications. Keep this guide on a second screen or printed.

0.2 The 2-Hour Session Agenda

Two hours is tight for an AI-assisted build of a full simulation. The plan below builds the highest-teaching-value steps LIVE and shows the independent enhancements as guided practice or pre-built walk-throughs. Times include prompting, reviewing, and discussion.

Segment	Time	Mode	Focus
Opening + workflow framing	12 min	—	Business story + how AI-assisted coding works (Sections 2 & 3). Set up <code>PROJECT_SPEC.md</code> on screen.
Step 1 — Data connectivity	14 min	BUILD LIVE	Hand the agent the plan prompt; it profiles <code>seed_data.py</code> and proposes an architecture. Then build the baseline <code>main.py</code> that loads and prints every table.
Step 2 — Order-to-recipe	16 min	BUILD LIVE	Prompt for recipe lookup + ingredient calculation + availability check. Review the no-matching-recipe edge case.
Step 3 — Fulfilment & deduction	16 min	BUILD LIVE	Prompt for Delivered/Not Delivered logic and inventory deduction. Catch the partial-deduction assumption.

Segment	Time	Mode	Focus
Step 4 — Cumulative deduction	16 min	BUILD LIVE	The marquee AI-coding moment: catch the reference-vs-copy state bug the agent tends to leave in.
Step 5 — Restock & expiry rules	14 min	SHOW / GUIDE	Threshold + expiry logic. Walk the prompt; emphasise the 'today's date' watch-out.
Enhancements (learner practice)	14 min	GUIDED	Learners drive one enhancement (partial fulfilment / predictive alerts / item disabling) using the 8-step playbook.
Wrap-up + Q&A	8 min	—	Lifecycle recap (Section 8) + AI-assisted-coding lessons + prepared Q&A (Section 9).

Total: ~120 minutes. If the agent is slow or a prompt needs several refinements, Step 5 and the enhancements are your buffer — they can be compressed or shown pre-built.

0.3 Live-Build vs Show-Pre-Built Strategy

You cannot generate a complete, refactored simulation live in two hours — agent generation plus your review takes real time, and live silence kills momentum. Use this strategy:

- **BUILD LIVE (Steps 1–4):** these are fast to generate and carry the richest AI-assisted-coding lessons — prompting, reviewing, catching the cumulative-state bug, iterating, and running tests. Do these in front of learners.
- **SHOW / GUIDE (Step 5 + enhancements):** restock rules and the three enhancements are ideal as a walk-through of the prompt plus guided learner practice. Display the already-generated code from your dry run and walk the prompt that produced it.
- **Strategy B fallback (have this ready):** keep your dry-run generated files saved. If a live prompt misbehaves badly, say 'let me show you the version I generated earlier with this same prompt', open the saved file, and keep moving. Never debug the agent silently for more than ~60 seconds in front of the room.
- Pre-open `PROJECT_SPEC.md` and the dry-run output folder before the session so the fallback is one click away.

0.4 What to Have Open Before Learners Arrive

- VS Code, with the project folder open and the Python interpreter selected.
- The AI agent panel, connected and signed in, with an empty chat ready.
- `PROJECT_SPEC.md` open in a tab (your persistent context).
- `seed_data.py` in the folder; your dry-run generated files saved in a sibling folder as the fallback.
- An integrated terminal open in the project folder, ready to run `python -m pytest -v`.
- This guide (second screen or printed) and a whiteboard for the opening timeline.

0.5 Opening Script — Your First Three Minutes

The first three minutes set the tone. If you prefer not to improvise, here is a complete script you can deliver word-for-word. Practice it once aloud.

OPENING SCRIPT (READ OR ADAPT)

“Good morning everyone. Today we're going to build something real — a simulation of how a cloud kitchen manages its inventory across multiple brands in a single busy day — and we're going to build the whole thing without writing code by hand. We'll prompt an AI coding agent, right here in VS Code, and our job will be to direct it, review what it produces, and catch its mistakes.”

“That second part is the real skill. Anyone can ask an AI to write code. The professional skill is knowing whether the code it gives back is correct — especially when small logic bugs, like an inventory count that doesn't carry over between orders, look fine on screen but quietly corrupt every number downstream. You'll see exactly that happen today.”

“Here's our case. A cloud kitchen runs several delivery-only brands from one kitchen, sharing staff, equipment, and one pool of ingredients. Margins are thin — after aggregator commissions, packaging, and food cost, a typical order nets only 5 to 15 percent. So when an ingredient silently runs out mid-shift, the kitchen keeps accepting orders it can't fulfil — lost revenue, refunds, and a hit to its platform ranking.”

[Whiteboard: draw the flow — Order arrives → Look up recipe → Check inventory → Deduct → Deliver or reject → Restock. Circle 'Check inventory'.] “Everything we build today lives right here — making sure the kitchen always knows what it has, what it's used, and what to reorder before it runs out.”

“We'll build the system in clear steps — connect the data, process an order against its recipe, fulfil and deduct stock, make the maths cumulative across orders, and finally automate restock and expiry rules. For each one, I'll show you the prompt, we'll read what the agent writes together, and we'll decide whether to trust it. Let's start by giving the agent its first instruction.”

0.6 Step-to-Step Highlights

Transitions are where live demos lose energy. One sentence each, memorise or keep visible:

Transition	Suggested highlight
Into Step 2	“The app can now see all its data. Time to teach it the kitchen's core rhythm — take an order, look up its recipe, and work out what it needs.”
Into Step 3	“We know what each order requires. Now let's actually serve it — deduct the stock, mark it delivered, and flag what we couldn't fulfil.”
Into Step 4	“One order works. But a real kitchen serves order after order from the same shelf — so the maths has to carry over. This is where the subtle bug hides.”
Into Step 5	“Our inventory is finally accurate. The last piece is the reorder brain — what's running low, what's about to expire, and how much to buy.”
Into enhancements	“You've watched me drive the agent through five steps. Now you take the wheel — same playbook, a new feature.”
Into wrap-up	“Let's zoom out — because the way we worked today is how you'll build almost any software with an AI from now on.”

0.7 Session-Day Run Sheet (condensed checklist)

When	Do
T-minus 30 min	Restart machine. Open VS Code in the project folder. Confirm the agent panel is connected and signed in. Select Python interpreter. Open PROJECT_SPEC.md and an integrated terminal. Open your saved dry-run files in a sibling folder (fallback). Zoom for readability. Close all notifications.
T-minus 5 min	Give the agent one warm-up prompt in a scratch file to confirm it responds, then discard it. Have whiteboard ready. This guide visible on second screen. Water within reach.
During	For each step: paste the PROMPT, read the agent's output aloud with the room, run the REVIEW checks, then run the code. Verify each 'What You Should See' box. Use segue lines (0.6) between steps. Park hard questions: 'Great question — I have a Q&A section on exactly that.'
If the agent misbehaves	Section 10 (Troubleshooting). One refinement prompt is fine and instructive; if a second fails, switch to your saved dry-run file ('here's the version I generated earlier') and move on. Never debug silently > 60 seconds.
Closing	Section 8 recap on screen → closing script → AI-assisted-coding lessons → Q&A (Section 9) → share PROJECT_SPEC.md and the prompts so learners can reproduce everything.

1. Jargon Buster — Read This First

Before walking learners through the demo, review this table. These are the terms that trip up beginners most often. Point learners here whenever you use one of these words.

Term / Jargon	What it actually means (plain English)
Cloud kitchen	A delivery-only food business where several virtual brands cook from one shared kitchen — same staff, equipment, and ingredient pool. Great for utilisation, hard for inventory.
Recipe (bill of materials)	The list of ingredients and quantities (in grams) a menu item needs. Each order item is 'exploded' into its ingredients before stock is checked.
Inventory / stock	How much of each ingredient the kitchen currently has, with an expiry date. The single shared pool every brand draws from.
Deduction	Subtracting the used grams from inventory once an order is fulfilled. Get this wrong across many orders and every downstream number is wrong.
Cumulative deduction	Each order checks stock against what's left after the previous order, not against the original starting stock. The heart of a realistic simulation.
Fulfilment status	Whether an order was 'Delivered' (all ingredients available) or 'Not Delivered' (something was missing). Stored per order in the Status table.
Stockout	An ingredient hitting zero mid-shift, so dependent menu items can no longer be made — the core failure this system prevents.
Par level	The target stock level to refill an ingredient back up to when restocking. Here, 10,000g.
Restock threshold	The low-stock trigger. If an ingredient falls to or below this level (here, 1,000g), it goes on the restock list.
Expiry window	How close to the expiry date an ingredient can get before it's flagged 'Expiring soon' (here, within 5 days of today).
Partial fulfilment	Serving the items in an order that CAN be made while separately flagging the ones that can't, rather than failing the whole order.
Par-level restocking	Topping every flagged ingredient back up to the par level (10,000g), rather than ordering an arbitrary amount.
Checkpoint restocking	Running the restock pass every N orders (a checkpoint), not after every single order — balancing recovery against efficiency.
Edge case	An unusual input the happy path forgets — an order item with no matching recipe, a None value, a negative quantity, an empty list. Where AI code quietly breaks.
Refactor	Rewriting code to remove duplication or improve structure without changing what it does. Run periodically because AI generates each prompt in isolation.
Unit test	A small automated check that confirms one function does exactly what it should. Runs in seconds; catches regressions instantly.

1.1 AI-Assisted Coding Terms

Because we build everything by prompting the agent, learners also need these terms:

Term / Jargon	What it actually means (plain English)
AI coding agent / agentic coding	An AI that doesn't just autocomplete a line — it takes a whole task ('build the order-processing function'), creates and edits files, and can run code, acting across your project like a junior developer you direct.
Prompt	The instruction you give the AI. The quality of the code you get back depends almost entirely on the quality of the prompt — being specific, giving context, and stating constraints.
PROJECT_SPEC.md	A plain-text file describing the project's goal, data, rules, and decisions already made. You point the AI at it so it keeps full context across many prompts instead of forgetting what you're building.
Context drift	As a chat grows, the AI's earlier instructions degrade or disappear. Re-anchoring from PROJECT_SPEC.md at the start of each session prevents it.
Diff / reviewing a diff	The set of changes the AI proposes, shown as additions and deletions. You read the diff and decide whether to accept, reject, or ask for changes — like reviewing a colleague's pull request.
Iterating / refinement prompt	Following up on the AI's first attempt with a correction ('good, but make the deduction cumulative and add comments'). Most real AI-assisted coding is a short back-and-forth, not one perfect prompt.
Hallucination	When an AI confidently produces something wrong or invented — a non-existent function, a wrong default, or a plausible-looking but incorrect approach. The reason you always review.
Validation hook	A check you ask the AI to build in to surface its own assumptions — e.g. inline comments stating what it hard-coded, what it's uncertain about, and what's incomplete.
Scope creep	The AI implementing tasks you didn't ask for yet. The phrase 'Do not implement any other tasks yet' in every implementation prompt keeps scope tight.
Generate → Review → Refine	The three-step loop behind every step: you prompt (generate), you read and check the output (review), you correct with one follow-up (refine). The reviewing is the job.

3. The AI-Assisted Coding Workflow — The Real Subject of This Session

The cloud kitchen simulation is the vehicle. The skill you are actually teaching is how to build software by directing an AI agent and critically reviewing its work. Spend real time on this section in the opening — it is the through-line that connects all five steps and the enhancements.

3.1 The Mental Model: The Director, Not the Typist

CONCEPT IN PLAIN ENGLISH

Before writing a single prompt, learners must internalise the correct mental model. AI is not a software engineer. It is a probabilistic generator — an advanced autocomplete trained on billions of lines of code. It predicts what looks plausible, not what is correct.

Think of it as a confident but forgetful junior developer: writes fast, has seen a lot of examples, sometimes gets details wrong, and forgets what you told it ten minutes ago. You play three roles throughout the project:

- **Navigator:** set the direction and goal. Break work into clear, small steps. Keep the big picture in focus.
- **Reviewer:** inspect every AI-generated output. Ask: does this do what I asked? Spot gaps, errors, or drift from intent.
- **Decision-Maker:** accept, reject, or redirect AI output. Own the final quality of the work. You are always accountable.

3.2 The Generate → Review → Refine Loop

Every step follows the same three-step loop. Teach it explicitly in Step 1, then name it each time so learners internalise the rhythm.

GENERATE

You write a clear prompt stating the goal, context, and constraints. The agent proposes (and often runs) code.

REVIEW

You read what it produced — out loud, with the room. Does it do what you asked? Is it correct? Does it handle the edge cases? This is the step beginners skip and professionals never do.

REFINE

If it's off, you don't hand-fix — you tell the agent what to change. One or two follow-ups usually get there. If it's right, you accept.

THE GOLDEN RULE TO REPEAT ALL SESSION

“You own every line the AI writes. If you can't explain it, you can't ship it.”

3.3 Anatomy of a Good Prompt

A weak prompt ('process the orders') gives generic, often wrong code. A strong prompt has four parts. Show learners this structure once, early:

- **GOAL — what you want:** 'Update main.py to process each order against its recipe and check inventory.'
- **CONTEXT — what the AI needs to know:** 'Data is in seed_data.py: Recipes, Inventory, Orders, Restock, Status. Quantities are in grams.'
- **CONSTRAINTS — the rules it must follow:** 'Deduction must be cumulative. Restock threshold = 1,000g, par level = 10,000g. Do not implement any other tasks yet.'
- **OUTPUT SHAPE — what you want back:** 'Add comments explaining each section, then add validation-hook comments listing assumptions and incomplete parts.'

AI WATCH-OUT — WHERE THE AGENT TENDS TO SLIP

The habit you are modelling: the AI acts confidently whether it's right or wrong. Its first draft of the inventory logic will look clean and run without error — yet quietly not carry stock over between orders, or fit its logic to a single order it saw. State your constraints, and review even when the output looks polished.

3.4 PROJECT_SPEC.md — Your AI's External Memory

AI has no persistent memory between sessions, and within a session earlier instructions degrade as the conversation grows. To prevent context drift, maintain a PROJECT_SPEC.md file and paste the relevant section at the start of each new session. A suitable version for today:

```
# Cloud Kitchen Inventory Simulation – Project Spec

## Goal
Simulate one operational day for a multi-brand cloud kitchen:
process orders in real time, track shared inventory and expiry,
and produce a prioritised restock plan.

## Data (seed_data.py)
Recipes (5 items), Inventory (14 ingredients + expiry),
Orders (5 orders, 2 brands), Restock (5 flagged), Status.
All ingredient quantities are in GRAMS.

## Hard rules
- Deduction is CUMULATIVE: each order checks stock remaining
  after the previous order, not the original stock.
- Update the final inventory table only after ALL orders run.
- Restock threshold = 1,000g. Par level = 10,000g.
- Expiry window = within 5 days of today → 'Expiring soon'.
- One centralised inventory shared by all brands.

## Conventions
- Build in one file, main.py; add one task per prompt.
- Add comments per section + validation-hook comments.
- Generate unit tests immediately after each function.
```

Session re-anchor prompt (start of each new session):

```
Here is our project context. We are building a Python cloud
kitchen inventory simulation using seed_data.py (Recipes,
Inventory, Orders, Restock, Status). We have completed Tasks
1-3. Task 4 (cumulative deduction) is the current task. Do
not modify any previously implemented functions.
```

Tell learners: 'Whether you keep this in a file or paste it into your first message, the point is the same — encode the cumulative rule and the thresholds ONCE so you're not re-typing them every prompt.'

3.5 The Prompt Templates Library

Four templates are reused throughout the entire project. Every prompt in the phase sections is built on one of these. Learn them once, apply them everywhere. Use Templates 1 and 2 before writing a single line of code — they are your architecture phase. Use Template 3 strictly one task at a time. Use Template 4 every time something breaks.

Template 1 — Planning Prompt

- **When to use:** At the very start of every new project or major component, before any code is written.
- **What it does:** Gets the AI thinking in components, not code. Gives you a shared architecture to reference throughout.

```
"I want to build [PROJECT]. Before writing any code, give me a high-level plan – what components do I need and what should I build first?"
```

Template 2 — Task Breakdown Prompt

- **When to use:** Once per component, after the plan from Template 1 is confirmed.
- **What it does:** Converts each component into individual, small, implementable tasks. Small tasks = small errors = fast debugging.

```
"Here is component [NAME] from our plan. Break it into individual implementation tasks. Keep each task small enough to implement in a single prompt."
```

Template 3 — Implementation Prompt

- **When to use:** For every individual task in the implementation phase. One task per prompt — never bundle tasks.
- **What it does:** Keeps scope tight. 'Do not implement any other tasks yet' prevents scope creep. Comments make output verifiable.

```
"Implement Task [N]: [DESCRIPTION]. Use Python. Keep the code simple and add comments explaining each section. Do not implement any other tasks yet."
```

Template 4 — Debugging Prompt

- **When to use:** Every time a test fails or code produces unexpected output. Never skip it and move on.
- **What it does:** Forces the AI to explain the root cause, not just patch the error — so you genuinely own the code.

```
"Here is the code and the error it produces: [PASTE CODE] / [PASTE ERROR]. Explain what is wrong and provide a corrected version with comments."
```

3.6 How to Review AI-Generated Code — The 5-Point Checklist

This is the checklist learners should leave with. Apply it visibly on every function the AI generates:

1. **Does it do what I asked?** Run it with 3 different inputs you can verify manually.
2. **Are there hidden assumptions?** Look for hard-coded values, assumed data types, or logic paths you didn't specify.
3. **Is anything hard-coded that should be a variable?** (e.g. restock thresholds, par levels, expiry windows.)
4. **Does it handle bad inputs?** What happens with None values, empty lists, or negative quantities?
5. **Is any part security-sensitive or business-critical?** (e.g. inventory deductions, expiry logic — the numbers that cost real money.)

4. The Business Case — What We're Building

CONCEPT IN PLAIN ENGLISH

A cloud kitchen is a delivery-only food business where multiple virtual brands operate from the same kitchen, sharing staff, equipment, and ingredients. This improves utilisation and revenue — but it creates real operational pain in inventory and recipe management.

Cloud kitchens run on tight margins. After aggregator commissions (typically 20–30% of order value), packaging, marketing, food cost, rent, labour, and utilities, the contribution margin on a typical order is often only 5–15%. Even small operational inefficiencies meaningfully dent profitability.

Our goal: build a Python simulation of a multi-brand kitchen's inventory and order management for a single operational day — one that processes orders in real time, tracks shared inventory and expiry, supports fulfilment decisions, and produces a prioritised restock plan. And today, we build it by prompting an AI coding agent.

Think of it like a fuel gauge for the kitchen. You don't wait until you're stranded — you watch the needle and refill before it hits empty. The simulation is that gauge.

Why This Problem Matters Financially

Business Fact	Details
The margin reality	After commissions, packaging, food, rent, labour and utilities, a typical order nets only 5–15% — so operational waste hits the bottom line directly.
Stockouts → lost sales	Every minute an ingredient is unavailable but its dependent menu items stay listed leads to order acceptances the kitchen can't fulfil — hurting revenue and platform ranking.
Food waste → lost margin	In poorly managed kitchens, 8–15% of food cost is thrown away monthly due to expiry. A recipe-linked, expiry-aware system can cut this significantly.
Experience → ranking	Rejected orders, missing items, and partial deliveries hurt aggregator ratings, reducing visibility and orders — a downward spiral.

The Key Problems We're Solving

- **Silent stockouts:** ingredients run out mid-shift, causing rejected orders.
- **Manual menu updates:** when an ingredient is unavailable, dependent items across brands and platforms must be disabled by hand.
- **No expiry tracking:** lack of freshness monitoring leads to waste and the risk of using expired ingredients.
- **Inefficient restocking:** poor consumption visibility causes over-ordering or under-ordering.
- **No structured procurement:** kitchens lack a prioritised, data-driven daily restock plan.

MENTOR SAYS

Start by setting the business scene. Say something like: 'Before we touch code, let's understand why this matters. A cloud kitchen keeps 5 to 15 cents on the dollar — so a single silent stockout, or a tray of expired stock, isn't a rounding error, it's the profit on dozens of orders. We're building the system that stops both.'

Draw a simple flow on the whiteboard: [Order arrives] → [Look up recipe] → [Check inventory + expiry] → [Deduct + Deliver, or Reject + flag] → [Restock at checkpoints]. Point out that the whole day is this loop repeated — and the maths has to stay correct across every pass.

Ask learners: 'Where do you think the money leaks in a kitchen like this?' Let a few answer before continuing.

DISCUSSION PROMPT — ASK THE LEARNERS

- Why is a shared inventory pool across brands both an advantage and a risk?
- If an ingredient runs out, why is disabling the affected menu items across every brand so important — and so easy to forget manually?
- Why is this a *simulation* worth building rather than just a spreadsheet the manager updates by hand?

PART 1 • CREATING THE ORCHESTRATOR

The goal is to ensure the application can “see” all the data it needs to work with. This layer is the foundation for the entire system. Each step follows the Build → Test → Inspect → Refine loop from the course.

Step 1 • Establishing Data Connectivity

BUILD LIVE WITH THE AGENT

Hand the agent the brief and the data, let it profile `seed_data.py` and propose an architecture — then build a baseline `main.py` that loads and prints every table so the app can 'see' all its data.

CONCEPT IN PLAIN ENGLISH

Before any logic, the application must be able to read every data structure it will work with. `seed_data.py` holds five tables: Recipes (5 items — Margherita Pizza, Chicken Burger, Caesar Salad, Pasta Alfredo, Chocolate Cake, each with ingredient quantities in grams), Inventory (14 ingredients with current stock and expiry dates), Orders (5 orders across two brands — Taco Bell and Subway), Restock (5 flagged ingredients with a reason), and Status (delivery status per order).

The lesson today: a well-briefed agent will propose a sensible architecture on its own — but your job is to verify the plan is grounded in what's actually in the file, and that the baseline reads every table correctly. You can only trust a foundation you can see.

Prompt 1 — Hand over the brief and ask for a grounded plan (Template 1)

I want to build a Python cloud kitchen inventory simulation using `seed_data.py`. It has 5 tables: Recipes, Inventory, Orders, Restock, Status (quantities in grams). Business goal: process a day of orders against shared inventory, track expiry, and produce a restock plan.

Before writing any code, profile `seed_data.py` and give me a high-level plan — what components do I need and what should I build first? Don't write the implementation yet.

This mirrors a real session: give the agent the framing plus the file and ask for a plan first. It will inspect the tables and come back with a component breakdown.

REVIEW THE AGENT'S OUTPUT — CHECK BEFORE ACCEPTING

- Confirm the plan references the actual five tables by name and treats Recipes as the link between an order item and its ingredients — not an invented schema.
- Confirm it proposes a sensible build order: data connectivity first, then order processing, then fulfilment, then cumulative logic, then restock rules.
- If the agent jumps straight to writing all the logic, rein it in: 'Plan only for now — we'll build Task 1 next, one task at a time.'

Prompt 1B — Build the baseline `main.py` (Template 3)

Implement Task 1: Create a baseline main.py that imports all data from seed_data.py. Write specific functions to load and print the contents of every table – Recipes, Inventory, Orders, Restock, and Status – to the console. Add comments explaining each function. Do not implement any other tasks yet.

Validation hook prompt (run right after):

After generating the code, add inline comments identifying: (1) any assumptions you made that I should verify, (2) any parts you are uncertain about, and (3) any sections that are incomplete or need follow-up.

Test generation prompt (immediately after):

Write unit tests for the load functions using Python unittest. Cover: successful import of all 5 tables, correct record count in each table, and correct data types for key fields (e.g. qty_grams is a number, expiry_date is a string).

WHAT YOU SHOULD SEE — VERIFY ON SCREEN

- main.py imports seed_data cleanly and runs with 0 errors.
- Five load/print functions, one per table, each with explanatory comments.
- Console shows all 5 Recipes, 14 Inventory rows with expiry dates, 5 Orders across Taco Bell and Subway, 5 Restock rows, and Status per order.
- Validation-hook comments flag at least the data-type assumptions (grams as numbers, expiry as strings).
- Unit tests run green: 5 tables imported, correct counts, correct types.

MENTOR SAYS

Name the loop from the very first step: 'Generate, Review, Refine. The agent generated a plan and a baseline; we read it against the actual tables and ran the tests. The reviewing is the job — I'll say those three words all session.'

On why we start here: 'This looks trivial — we're just printing tables. But if the app can't reliably see its data, nothing built on top of it can be trusted. Foundations first.'

AI WATCH-OUT — WHERE THE AGENT TENDS TO SLIP

Watch for invented field names. If the agent assumes a column that isn't in seed_data.py (a common hallucination), the print will fail or show blanks — catch it by reading its output against the real file.

Watch for bundled scope. A generic 'set up the project' prompt often tempts the agent to start processing orders too. Keep Task 1 to loading and printing only — the discipline you set here pays off in Step 4.

DISCUSSION PROMPT — ASK THE LEARNERS

- The agent proposed an architecture on its own. How would you verify that plan is grounded in the real data rather than just plausible-sounding?
- Why do we insist on one task per prompt instead of asking for the whole simulation at once?
- What could go wrong later if the baseline silently misreads one table now?

PART 2 • DEFINING THE APPLICATION FLOW

Now that the data is connected, we define the “rhythm” of the kitchen: taking an order, checking ingredients, and updating the status.

Step 2 • Order-to-Recipe Processing

BUILD LIVE WITH THE AGENT

Prompt the agent to pull an order, look up each item's recipe, calculate total ingredients required, and check availability — then review the no-matching-recipe edge case it tends to skip.

CONCEPT IN PLAIN ENGLISH

Each menu item is linked to a recipe. When an order arrives, the system looks up the recipe for each item, multiplies by the ordered quantity to get total grams required, and checks the Inventory table for sufficient, unexpired stock.

The subtle trap here is the missing recipe. What happens when an order references an item that has no recipe entry? A happy-path implementation will crash or silently skip it. Real kitchens need this handled gracefully — that's the review focus.

Prompt 2 — Task breakdown, then implement (Templates 2 + 3)

First break the component down:

Here is component 'Order Processing' from our plan. Break it into individual implementation tasks. Keep each task small enough to implement in a single prompt.

Then implement the first task:

Implement Task 2: Update main.py to process orders from the Orders table. For each order: 1. Look up the recipe for each item. 2. Calculate the total grams of ingredients required (recipe qty x order quantity). 3. Check the Inventory table to see if all required ingredients are present in the necessary quantities. Add comments explaining each step. Do not implement any other tasks yet.

Validation hook prompt:

After generating the code, add inline comments identifying: (1) any assumptions about how recipe lookup works, (2) what happens if an order item has no matching recipe, and (3) any sections that are incomplete.

Test generation prompt:

Write unit tests for the order-to-recipe lookup using unittest. Cover: a valid order item with a matching recipe, an order item with no matching recipe (must handle gracefully, not crash), and correct total-grams calculation for an order with quantity 2.

REVIEW THE AGENT'S OUTPUT — CHECK BEFORE ACCEPTING

- Confirm the quantity multiplication is correct: recipe grams × ordered quantity, not just the recipe grams. An order of 2 must require double.
- The key review: the **missing-recipe path**. Does the code handle an item with no recipe gracefully, or does a `KeyError` take down the whole run? If it crashes, refine.
- Confirm the availability check compares against Inventory correctly and doesn't yet deduct anything — deduction is Step 3.

WHAT YOU SHOULD SEE — VERIFY ON SCREEN

- For each order, the console lists required ingredients and total grams per ingredient.
- An order with quantity 2 shows exactly double the recipe amounts.
- An order item with no matching recipe is reported and skipped, not crashed on.
- Availability is reported (enough / not enough) without any stock being changed yet.
- Unit tests green, including the no-matching-recipe case.

MENTOR SAYS

On the missing recipe: 'This is the kind of case beginners forget and the AI often skips — what if someone orders an item we have no recipe for? On the happy path everything looks fine. The professional habit is to ask the AI, out loud, what happens on the unhappy path — then check its answer.'

AI WATCH-OUT — WHERE THE AGENT TENDS TO SLIP

The agent frequently forgets to multiply by order quantity, computing the recipe's base grams regardless of how many were ordered. Verify with an order of 2. It also tends to write the missing-recipe branch only if you explicitly ask via the validation hook — which is exactly why we always ask.

DISCUSSION PROMPT — ASK THE LEARNERS

- Why is the missing-recipe case 'sneaky' — why does it look fine right up until it isn't?
- The agent computed required grams. Why is it important that this step does NOT yet deduct from inventory?
- How would you write a test that proves an order of quantity 2 is handled correctly?

Step 3 • Fulfilment & Inventory Deduction

BUILD LIVE WITH THE AGENT

Prompt the agent to mark orders Delivered or Not Delivered, deduct stock when fulfilled, and log missing items to Restock — then review the partial-deduction assumption.

CONCEPT IN PLAIN ENGLISH

Now we act on the availability check. If all ingredients for an order are available, mark it 'Delivered' in the Status table and deduct the used grams from Inventory. If anything is missing, mark it 'Not Delivered', record the reason, and add the missing items to the Restock table.

The critical question: what if only SOME ingredients are available? Does the code deduct the ones it has (leaving the order half-served and the books wrong), or does it treat the order as all-or-nothing? For now the rule is all-or-nothing per order — partial fulfilment is a later enhancement. Verify the agent doesn't silently deduct partial stock.

Prompt 3 — Implement fulfilment logic (Template 3)

Implement Task 3: Add fulfilment logic. If all ingredients are available, mark the order 'Delivered' in the Status table and deduct the used grams from Inventory. If any ingredient is missing, mark the order 'Not Delivered', record the reason, and add the missing items to the Restock table. Treat each order as all-or-nothing for now. Add comments. Do not implement any other tasks yet.

Validation hook prompt:

After generating the code, add inline comments identifying: (1) what happens if the same ingredient is missing from two consecutive orders, (2) any assumption about whether partial stock can be deducted, and (3) any incomplete sections.

Test generation prompt:

Write unit tests for the fulfilment function. Cover: an order where all ingredients are available (expect Delivered), an order where one ingredient is missing (expect Not Delivered, missing item logged to Restock), and correct deduction from inventory after a delivery.

REVIEW THE AGENT'S OUTPUT — CHECK BEFORE ACCEPTING

- The key review: **no partial deduction**. A Not-Delivered order must NOT have deducted the ingredients it did have. If the code deducts as it goes and then hits a missing item, the inventory is now silently wrong. Confirm it checks full availability first, then deducts.
- Confirm Delivered orders deduct the correct grams and Not-Delivered orders write a clear reason to Status.
- Confirm missing items land in the Restock table, not just printed to the console.

WHAT YOU SHOULD SEE — VERIFY ON SCREEN

- Each order ends as Delivered or Not Delivered in the Status table, with a reason where relevant.
- Inventory decreases only for Delivered orders, by exactly the required grams.
- A Not-Delivered order leaves inventory untouched (no partial deduction) and adds its missing items to Restock.
- Unit tests green across the delivered, not-delivered, and deduction cases.

MENTOR SAYS

On partial deduction: 'Watch this one carefully. The natural way to write this — loop through ingredients, subtract each as you go — has a nasty bug: if you run out halfway, you've already eaten into stock for an order you can't fulfil. The correct pattern is check-all-then-deduct. I'll ask the agent which one it did, and I'll verify by reading the code.'

AI WATCH-OUT — WHERE THE AGENT TENDS TO SLIP

The classic bug here is deduct-while-checking, which corrupts inventory on any Not-Delivered order. The agent's first draft often does exactly this because it's the more obvious loop. Insist on 'verify full availability before deducting anything', and confirm it in the diff.

DISCUSSION PROMPT — ASK THE LEARNERS

- Why is deduct-as-you-go dangerous even though it looks simpler?
- If an order is Not Delivered, what exactly should have changed in the data — and what should NOT have changed?
- Where would you add a validation check to prove no partial deduction ever happens?

PART 3 • REFINING THE LOGIC FOR CORRECT OUTPUT

The final part ensures the inventory maths is accurate across multiple orders and that restocking triggers correctly on the health rules.

Step 4 • Cumulative Inventory Deduction

BUILD LIVE — THE MARQUEE MOMENT

Prompt the agent to make deduction cumulative across orders — then catch the reference-vs-copy state bug it tends to leave in. This is the single most important AI-coding lesson in the session.

CONCEPT IN PLAIN ENGLISH

So far each order may have been checked against the ORIGINAL starting stock. In a real day, Order 2 must be checked against whatever remains after Order 1 was served. Deduction has to be cumulative: the inventory is a single, evolving state that every order both reads from and writes to.

Two subtle things go wrong here. First, whether the code mutates one shared inventory object or accidentally works on fresh copies each time (so deductions never persist). Second, when to publish the 'final' inventory table — only after ALL orders have processed, to show the true remaining stock. This is exactly the kind of bug that runs without error and quietly produces wrong numbers.

Prompt 4 — Make deduction cumulative (Template 3)

Implement Task 4: Refine the inventory logic to be cumulative. Order 2 must check availability against the quantity remaining after Order 1 was served, and so on. Maintain one working inventory state across all orders. Update the final inventory table only after all orders in the list have been processed, so it shows the true remaining stock. Add comments. Do not implement any other tasks yet.

Validation hook prompt — this one surfaces the bug:

After generating the code, add inline comments identifying:
(1) whether a deep copy or a reference is used for the working inventory state, and confirm deductions persist across orders,
(2) what happens if two orders compete for the same ingredient, and (3) any incomplete sections.

Test generation prompt:

Write unit tests for cumulative deduction. Cover: two orders consuming the same ingredient (verify the combined deduction), an order that fails because a prior order consumed the remaining stock, and the final inventory state matching expected remaining quantities.

REVIEW THE AGENT'S OUTPUT — THE KEY REVIEW OF THE SESSION

- **State handling (most important):** confirm the code carries ONE evolving inventory across all orders. If it re-reads the original seed each iteration, or copies the inventory afresh per order, deductions vanish and every order sees full stock. Prove it with the two-orders-same-ingredient test.
- Confirm the final table is published only after all orders run — not mid-loop, which would show a half-processed state.
- Confirm an order that should fail because a prior order drained the stock actually fails — this is the proof cumulative logic is real.

WHAT YOU SHOULD SEE — VERIFY ON SCREEN

- Two orders consuming the same ingredient show the ingredient reduced by the COMBINED amount, not just the last order's amount.
- An order that would have passed against original stock now correctly fails when a prior order used the remaining quantity.
- The final inventory table appears once, after all orders, with true remaining quantities.
- Validation-hook comment explicitly states reference-vs-copy and confirms persistence.
- Unit tests green, including the 'fails due to prior consumption' case.

MENTOR SAYS

Let this land as the headline moment: 'Here's the trap. The agent will write something that runs perfectly and looks right — but if it's working on a fresh copy of the inventory each order, none of the deductions stick, and every order thinks the shelf is full. It's invisible on the happy path. The two-orders test is what exposes it. That one test is worth more than any line of code I could type.'

Name the loop one more time: 'Generate, Review, Refine. It generated cumulative logic; we reviewed the state handling with a targeted test; if it was copying instead of carrying state, one refinement prompt fixes it.'

AI WATCH-OUT — WHERE THE AGENT TENDS TO SLIP

This is real and common: the agent's first cumulative implementation often re-initialises the inventory from seed inside the order loop, or deep-copies it per order — so deductions never accumulate. It runs with 0 errors and prints plausible numbers. Only a test that spans two orders against the same ingredient reveals it. Also watch for it updating the 'final' table inside the loop, showing a mid-run state as if it were final.

DISCUSSION PROMPT — ASK THE LEARNERS

- The code ran with no errors but the numbers were wrong. What does that tell you about trusting AI code that 'works'?
- Why does a test spanning two orders catch a bug that a single-order test never would?
- What's the difference between mutating a shared object and working on a copy — and why does it matter for a running total?

Step 5 • Restock & Expiry Rules

SHOW / GUIDE — WALK THE PROMPT

Automate restock reasons from expiry dates and low-stock thresholds. Walk the prompt line by line; the 'today's date' handling is the watch-out that quietly breaks a simulation.

CONCEPT IN PLAIN ENGLISH

The final logic layer turns the post-order inventory into a prioritised reorder plan. Three rules, applied to the final stock: an ingredient within 5 days of expiry (relative to today) is 'Expiring soon' and set to restock 10,000g; an ingredient at 0 is 'Out of stock' (10,000g needed); an ingredient at or below 1,000g is 'Running low on stock', topped up to reach 10,000g.

An ingredient can be flagged for more than one reason at once (low AND expiring) — the audit comments must preserve both. And crucially, the 'today' the code compares against must be correct for the simulation, or every expiry decision is wrong.

Prompt 5 — Implement restock and expiry rules (Template 3)

```
Implement Task 5: Update the restock logic with these rules,
applied to the final inventory: 1. If an ingredient is within 5
days of expiry (relative to today's date), mark 'Expiring soon'
and set restock quantity to 10,000g. 2. If final stock is 0,
mark 'Out of stock' (10,000g needed). 3. If stock is <= 1,000g,
mark 'Running low on stock' and calculate the amount needed to
reach 10,000g. Allow an ingredient to be flagged for both low
stock and expiry, preserving both audit comments. Add comments.
Do not implement anything beyond this task.
```

Validation hook prompt — the one that matters here:

```
After generating the code, add inline comments identifying:
(1) how today's date is obtained – verify this is correct for
the simulation, (2) whether an ingredient can be flagged for
both low stock and expiry simultaneously, and (3) any incomplete
sections.
```

Test generation prompt:

```
Write unit tests for the restock logic. Cover: an ingredient
expiring within 5 days (expect 'Expiring soon', 10,000g), an
ingredient with 0 stock (expect 'Out of stock', 10,000g), an
ingredient with 500g (expect 'Running low', 9,500g needed), and
an ingredient with adequate stock and no expiry issue (expect
not flagged).
```

REVIEW THE AGENT'S OUTPUT — CHECK BEFORE ACCEPTING

- **Date handling:** confirm how 'today' is derived. If the simulation assumes a fixed reference date but the code uses the live system clock, every expiry flag will drift as the calendar moves. Decide and state which one you want.

- Confirm multi-reason flagging works: an ingredient both low and expiring must carry both flags and both comments, not just the first one hit.
- Confirm the top-up maths: a 500g ingredient needs 9,500g to reach the 10,000g par level — not a flat 10,000g.

WHAT YOU SHOULD SEE — VERIFY ON SCREEN

- Expiring within 5 days → 'Expiring soon', 10,000g
- Stock = 0 → 'Out of stock', 10,000g
- Stock = 500g → 'Running low on stock', 9,500g needed
- An adequately stocked, in-date ingredient is not flagged at all.
- An ingredient that is both low and expiring carries both reasons in its audit comment.
- Unit tests green across all four cases.

MENTOR SAYS

On the date: 'This is where a demo silently rots. If the code reads the real clock and your expiry dates were set months ago, everything reads as expired — or nothing does. Ask the agent, out loud, "how are you getting today's date?" and make a deliberate choice for the simulation.'

On multi-reason flags: 'A single ingredient can be in trouble two ways at once. The lazy implementation stops at the first reason it finds. We want the full picture — both flags, both comments — because a buyer needs to know everything about that item.'

AI WATCH-OUT — WHERE THE AGENT TENDS TO SLIP

Two recurring slips: (1) using `datetime.today()` when the simulation needs a fixed reference date — confirm which, or expiry logic is meaningless; (2) short-circuiting on the first matching restock reason so an ingredient that is both low and expiring only shows one flag. Insist on multi-reason flagging and verify the audit comments.

DUPLICATION CHECK — RUN AFTER STEPS 2–5

The agent generates each prompt in isolation and has no global view of your codebase — duplicate logic accumulates silently. After Step 5, run this refactoring prompt before the enhancements:

Review `main.py` for duplicated logic — particularly any inventory lookup, availability check, or restock-flagging logic that appears more than once. Suggest how to refactor to eliminate duplication while keeping behaviour identical. Do not change any function signatures yet.

DISCUSSION PROMPT — ASK THE LEARNERS

- Why is 'how do you get today's date?' one of the most important questions in this whole build?
- Why do we top up to a par level rather than ordering a fixed amount every time?

- Why run a duplication check now rather than at the very end?

6. The Debugging Loop — Template 4 in Practice

When any test fails or any step produces unexpected output, do not skip ahead. Use this structured loop — it is how you turn a broken run into a genuine teaching moment.

1. **Read the error message carefully** — before anything else.
2. **Understand what it's saying** — what line, what type of error, what variable?
3. **Use Template 4** — paste both the code AND the full error.
4. **Ask the AI to explain the root cause**, not just give a fix.
5. **Apply the fix and re-run ALL tests** — not just the failing one.
6. **If still failing**, paste the new error and repeat.

Here is the code and the error it produces: [PASTE FUNCTION CODE] / [PASTE FULL ERROR MESSAGE]. Explain what is wrong and why. Then provide a corrected version with comments explaining what was changed and why.

MENTOR SAYS

'When something breaks live, don't panic and don't hand-fix. Read the error to the room, ask the agent for the root cause, and apply its fix — then re-run everything. Watching a real bug get diagnosed is more valuable than a demo that never breaks. The one rule: never debug silently for more than about a minute. If the second attempt misses, switch to your saved dry-run file.'

7. Session Exercise — Application Enhancements

CONCEPT IN PLAIN ENGLISH

This is where learners take the wheel. Having watched you drive the agent through five steps, they now apply the SAME 8-step playbook independently for one enhancement. Do not let them skip planning or jump straight to code.

Each enhancement follows: **Plan → Task Breakdown → Implement one task → Validate hooks → Test → Debug → Refactor.**

Enhancement 1 — Partial Fulfilment Logic

Modify order processing so that if a subset of items in an order can be fulfilled with remaining inventory, those items are delivered and marked separately rather than failing the entire order.

Starting prompt (Template 1):

I want to add partial fulfilment logic to the cloud kitchen simulation. Before writing any code, give me a plan – what components need to change and what should I build first?

Enhancement 2 — Predictive Stockout Alerts

Enhance the system to calculate the current per-order consumption rate and forecast which ingredients will run out within the next batch of orders, even if they are currently in stock.

Starting prompt (Template 1):

I want to add predictive stockout alerts to the cloud kitchen simulation. Before writing any code, give me a plan – what components do I need and what should I build first?

Enhancement 3 — Dynamic Item Disabling

Implement a function that automatically identifies all menu items across all brands linked to a depleted ingredient and marks them 'unavailable' to prevent future order rejections.

Starting prompt (Template 1):

I want to implement dynamic item disabling in the cloud kitchen simulation. Before writing any code, give me a plan – what components do I need and what should I build first?

REMINDER FOR LEARNERS

After each implementation prompt, request validation hooks. After each function, request unit tests. Use Template 4 whenever tests fail. Re-anchor from PROJECT_SPEC.md if the session gets long.

DISCUSSION PROMPT — ASK THE LEARNERS

- Enhancement 1 changes the meaning of an order's status. What new statuses or fields does partial fulfilment force you to add?

- For Enhancement 2, what assumption about consumption rate is the forecast built on — and when would it be wrong?
- For Enhancement 3, why is disabling items across ALL brands (not just one) the whole point?

8. Putting It All Together — The Full Picture

CONCEPT IN PLAIN ENGLISH

Zoom out. We started with a business problem — thin margins bleeding out through stockouts and waste — and a single `seed_data.py` file. We ended with a working simulation that processes a day of orders against shared inventory, keeps the maths correct across every order, and produces a prioritised, expiry-aware restock plan. And we built every line of it by prompting the agent, reviewing its output, and testing it.

Two lifecycles ran in parallel today: the **build lifecycle** (data → order flow → fulfilment → cumulative logic → restock rules) and the **AI-assisted coding loop** (generate → review → refine) repeated at every step. The second is the transferable skill — it works for any software, not just this simulation.

Step	Name	What it delivered
1	Data Connectivity	Baseline <code>main.py</code> that loads and prints all five tables. Used the AI to propose the architecture, then verified it against the real data.
2	Order-to-Recipe	Recipe lookup + total-grams calculation + availability check. Caught the no-matching-recipe edge case via a validation hook.
3	Fulfilment & Deduction	Delivered / Not Delivered status, correct deduction, missing items to Restock. Prevented the deduct-while-checking partial-deduction bug.
4	Cumulative Deduction	One evolving inventory state across all orders. Caught the reference-vs-copy bug that runs clean but produces wrong numbers — the marquee lesson.
5	Restock & Expiry Rules	Expiring-soon / out-of-stock / running-low flags with par-level top-up and multi-reason audit comments. Verified the 'today's date' handling.
Ext.	Enhancements	Partial fulfilment, predictive alerts, dynamic item disabling — driven by learners using the full 8-step playbook.

MENTOR SAYS

Close on the meta-skill: 'Every software project follows some version of this cycle — and increasingly, you'll build each piece the way we did today: prompt, review, refine. The simulation we built is disposable. The way of working is the asset.'

Name the discipline one more time: 'The AI generated hundreds of lines today. Our value was never typing — it was knowing what to ask for, spotting the cumulative-state bug, insisting on the edge cases, and reading the tests. That judgement is what you're being paid for in an AI world.'

Ask the group: 'What was the most surprising moment?' Common answer: code that ran perfectly yet carried a silent bug — and how a one-sentence refinement fixed it.

DISCUSSION PROMPT — ASK THE LEARNERS

- If you had to explain this whole project to a kitchen owner in three sentences, what would you say?
- Where did human judgement matter most today — where would trusting the AI blindly have sunk the project?
- What would need to change before this simulation could run against a real kitchen's live data? List at least five steps.

9. Deeper Insights

Question	Answer
“Why simulate a single day instead of a live system?”	A one-day simulation is enough to expose every core problem — cumulative deduction, expiry, restock — without the complexity of concurrency, databases, or real-time feeds. Once the logic is correct here, wiring it to live data is an engineering task, not a logic one.
“Why restock every N orders instead of after every order?”	Restocking after every order is unrealistic and inefficient — no kitchen reorders continuously. Checkpoint restocking (every N orders) mirrors how real replenishment batches work and keeps order processing and replenishment as separate, clean workflows.
“Why top up to a par level instead of a fixed reorder amount?”	A par level (10,000g) targets a consistent stock ceiling regardless of how depleted an item is. A 500g item needs 9,500g; an out-of-stock item needs 10,000g. Fixed reorder amounts either over- or under-shoot depending on where you started.
“Why must deduction be cumulative?”	Because a real shelf is shared. If each order checked the original stock, ten orders could each 'succeed' against the same 1kg of cheese. Cumulative deduction is what makes the simulation reflect reality — and it's the step where AI code most often goes subtly wrong.
“Why check full availability before deducting?”	To avoid partial deduction on a failed order. Deducting as you go means a Not-Delivered order still eats into stock for ingredients it did have — corrupting the books for every order after it. Check-all-then-deduct keeps the data honest.
“Why can one ingredient have two restock reasons?”	An item can be both running low AND expiring soon. A buyer needs the full picture, so both flags and both audit comments are preserved rather than short-circuiting on the first reason found.
“Why write unit tests for a simulation?”	Tests are how you trust code you didn't type by hand. They run in seconds, prove each function does what it should, and catch regressions instantly — which matters doubly for AI-generated code that looks right but may carry a silent bug.

9.1 Questions About the AI-Assisted Coding Itself

Question	Answer
“Can't the AI just build the whole thing without us?”	It can produce something that runs — but its first cumulative-deduction draft may quietly work on copies of the inventory, so no deduction sticks, and every number downstream is wrong. It took a human test to catch it. The AI optimises for what you ask; the human owns the spec, the review, and the judgement. The AI is the fast hands, not the engineer.
“How do I know the AI's code is actually correct?”	Three layers: read it (can you explain every line?), check it against the rules (the 5-point checklist in Section 3.6), and test it (unit tests after every function). And read the tests too — an AI can write a test that asserts nothing. Correctness comes from review and tests, never from trust.
“What if the agent gives different code each time I ask?”	It often will — that's expected. What stays constant is your spec and your checks: cumulative deduction, no partial deduction, the right thresholds, the passing tests. You're not memorising one correct output; you're enforcing the constraints any correct output must satisfy. That's why PROJECT_SPEC.md matters.
“Is learning to hand-code obsolete now?”	The opposite. You can only review what you understand. Everything that made today work — spotting the state bug, knowing to check the edge cases, recognising a no-op test — comes from understanding the code. The AI raises the value of fundamentals; it doesn't remove it.
“Why build some steps live but guide others?”	Time and teaching value. The core logic steps carry the richest lessons and are fast to generate, so we do them live. The enhancements are ideal as learner practice. Knowing which work to do live versus prepare is itself a skill of running an AI-assisted workflow.
“What do I put in a prompt to avoid these mistakes?”	State the constraints the AI can't infer: deduction is cumulative, check before deducting, the thresholds and par level, one task at a time, and ask it to add a validation hook. Put the durable rules in PROJECT_SPEC.md so you don't repeat them every prompt. Specificity is the whole game.

10. Live-Demo Troubleshooting

Things that can go wrong mid-demo, and how to recover.

Symptom	Cause	Fix / Recovery line
Agent panel won't connect / not signed in	Auth expired, or network/extension issue	Don't troubleshoot auth live. Switch to your saved dry-run files: 'Here's the build I generated earlier with these exact prompts.' Re-auth in the break.
Agent generates wrong or buggy code	Prompt under-specified, or normal AI variance	This is a teaching moment, not a failure. Name what's wrong, give ONE refinement prompt. If the second attempt also misses, open your saved correct file. Lesson: 'review catches this.'
Agent is slow to respond	Large request or service latency	Fill the gap deliberately — explain what you asked for and what you'll check when it returns. Never stare at a spinner in silence.
Code won't run (ImportError / NameError)	Missing import or a reference to something not yet created	Read the error with the room, then prompt: 'That failed with <error> — add the missing import and re-run.' The agent usually self-corrects in one turn. Models the real workflow honestly.
Agent edits the wrong part / sweeping changes	Ambiguous prompt scope; agent took initiative	Reject the diff rather than accept-and-fix. Re-prompt with explicit scope: 'Only modify the Step 2 functions in main.py; leave the rest untouched.' Reviewing the diff before accepting is the safeguard — show that.
ModuleNotFoundError	Package not installed in the active environment	Run <code>pip install <package></code> in the integrated terminal. Prevention: confirm your interpreter and any needed packages in Section 0.1.
Inventory numbers look wrong but no error	The cumulative-state bug (Step 4) — deductions not persisting	This is the marquee lesson, live. Run the two-orders-same-ingredient check, show the deduction didn't accumulate, and prompt the fix. Don't rush past it.
Everything reads as expired / nothing expires	Code uses the live clock against old seed dates	The Step 5 date watch-out. Ask the agent how it gets 'today', then set a fixed reference date for the simulation and re-run.
pytest reports a failure or error	A real bug in generated code, or environment mismatch	If it's a quick fix, prompt the agent to repair it — an honest demo of the workflow. If it's stuck, switch to your saved green dry-run and move on; debug after the session.
Live output differs slightly from this guide	Normal AI variance across runs	Quote your live number. Say: 'The AI generates fresh each time, so small variations are normal — the logic and the checks are what matter, and those are unchanged.'
seed_data.py fails to load	Wrong working directory, or file renamed	Confirm seed_data.py is in the project folder (<code>ls / dir</code> in the terminal). Tell the agent the exact filename and that it lives in the working directory.

11. Known Limitations — What to Tell Learners

Part of being a credible engineer is knowing and communicating the limits of your own work. Teach learners to flag limitations proactively — it builds trust.

Limitation	Plain-English explanation	What would fix it
Single-day, single-run snapshot	The simulation processes one fixed list of orders once. It doesn't model a rolling multi-day operation or demand that changes hour to hour.	Extend to multi-day runs with per-day order streams and carry-over inventory and expiry between days.
No concurrency	Orders are processed one after another. A real kitchen takes orders in parallel across brands, which can race for the same ingredient.	Add a queue and locking model, or simulate concurrent arrivals and resolve contention explicitly.
Fixed par level and thresholds	The 10,000g par level and 1,000g threshold are the same for every ingredient, regardless of how fast each is used.	Set per-ingredient thresholds driven by consumption rate (ties into the predictive-alerts enhancement).
Seed data, not live data	We run on a small hand-authored seed_data.py. A real kitchen has far more items, brands, and messy real-world data.	Connect to the POS / aggregator feeds and inventory system once a data pipeline exists.
Restock is instantaneous	Flagged ingredients are assumed refilled to par at the checkpoint. Real restocking has lead time, supplier limits, and cost.	Model supplier lead times, minimum order quantities, and a procurement budget constraint.

12. Mentor Quick-Reference Cheat Sheet

Tear-out reference for the key numbers and decisions during the live demo.

Key fact / number	What to say about it
5 data tables	Recipes, Inventory, Orders, Restock, Status — all in seed_data.py. The one file you provide; everything else is generated.
5 recipes / 14 ingredients	Margherita Pizza, Chicken Burger, Caesar Salad, Pasta Alfredo, Chocolate Cake; 14 ingredients with stock and expiry. Quantities in grams.
5 orders / 2 brands	Orders span Taco Bell and Subway from one shared kitchen and one shared inventory pool.
Restock threshold = 1,000g	At or below this, an ingredient is 'Running low on stock' and topped up toward par.
Par level = 10,000g	The target refill level. A 500g item needs 9,500g; an out-of-stock item needs 10,000g.
Expiry window = 5 days	Within 5 days of today → 'Expiring soon', restock 10,000g. Verify how 'today' is derived (Step 5 watch-out).
Cumulative deduction	Each order checks stock remaining after the previous order. The marquee bug: AI code that works on copies never accumulates — catch it with the two-orders test.
Check-all-then-deduct	Never deduct partial stock on a failed order. Verify full availability before subtracting anything.
Multi-reason restock flags	An ingredient can be both low AND expiring — preserve both flags and both audit comments.
The 4 prompt templates	Plan (T1) → Task Breakdown (T2) → Implement one task (T3) → Debug (T4). Say the sequence aloud.
The 5-point review checklist	Does it do what I asked? Hidden assumptions? Hard-coded values? Bad-input handling? Business-critical logic? Apply to every function.
Generate → Review → Refine	The loop behind every step. Generate = prompt; Review = read + check against Section 3.6; Refine = one-sentence correction. Golden rule: you own every line the AI writes.
Steps LIVE vs GUIDE	LIVE: 1–4 (fast, high-teaching-value, includes the marquee bug). GUIDE: Step 5 + enhancements (walk the prompt, learner practice). Section 0.3.
The cumulative-state lesson	Step 4: the agent's first draft runs clean but may not carry stock across orders. Review even polished output; a targeted test is the safety net.

End of Mentor Demo Guide · For Instructor Use Only